Flutter Quiz App

Complete Project Documentation

Architecture · Files · Functions · Firebase + Hive Implementation

---

1. App Overview

Quiz App is a Flutter application that allows two types of users to interact with a quiz system:

- Administrators — manage quiz categories and questions
- Students — register, log in, and take quizzes

The app uses a dual-storage architecture: Firebase Firestore as the primary cloud database and Hive as a local offline fallback. Every write operation saves to both simultaneously, so the app continues to work without internet.

Key Features

- Admin authentication with password reset
- Student registration via Firebase Auth with mandatory email verification
- Category management (full CRUD)
- Question management with multiple-choice answers
- Real-time quiz scoring
- Score history tracking per student
- Session management with persistent auto-login
- SHA-256 password hashing for admin and offline student auth
- Online-first reads with silent offline fallback

2. Technology Stack

Component	Technology	Purpose
UI Framework	Flutter + Material Design 3	Cross-platform UI
Cloud Database	Firebase Firestore	Primary persistent storage (online)
Authentication	Firebase Auth	Student registration, login, email verification
Local Database	Hive (NoSQL)	Offline fallback storage
Password Security	SHA-256 (crypto)	Secure admin / offline student password storage
ID Generation	UUID package	Unique identifiers for categories
State Management	StatefulWidget local state	Form and UI state

3. Project Structure

```
quiz_app/
├── lib/
│   ├── main.dart                          # App entry point
│   ├── database/
│   │   └── database_helper.dart           # CRUD — Firestore + Hive (Singleton)
│   ├── models/
│   │   ├── student.dart                   # Student & ScoreHistory models
│   │   ├── category.dart                  # Category model
│   │   └── question.dart                  # Question model
│   ├── screens/
│   │   ├── home_screen.dart
│   │   ├── admin_login_screen.dart
│   │   ├── admin_dashboard_screen.dart
│   │   ├── category_management_screen.dart
│   │   ├── add_category_screen.dart
│   │   ├── question_management_screen.dart
│   │   ├── add_edit_question_screen.dart
│   │   ├── student_login_screen.dart
│   │   ├── student_registration_screen.dart
│   │   ├── student_dashboard_screen.dart
│   │   ├── student_category_selection_screen.dart
│   │   ├── student_quiz_screen.dart
│   │   └── student_result_screen.dart
│   └── utils/
│       ├── session_manager.dart           # Session persistence (Hive)
│       └── password_utils.dart            # SHA-256 hashing
└── pubspec.yaml
```

4. Architecture & Design Patterns

4.1 Singleton Pattern

DatabaseHelper and SessionManager each expose a single global instance (DatabaseHelper.instance, SessionManager.instance). DatabaseHelper maintains two live connections: one to Firestore (_firestore) and one to Firebase Auth (_auth).

4.2 Online-First, Offline-Fallback Pattern

Every read operation tries Firestore first. If Firebase is unavailable — no internet, permission error, or timeout — the exception is silently caught and execution falls through to the local Hive box. Every write operation saves to both stores simultaneously.

```
Read Operation:
  Try Firestore ──▶ success ──▶ return data
        │
        │  fails (any exception — silent catch)
        ▼
  Try Hive ────────▶ success ──▶ return data
        │
        │  not found
        ▼
  Return null / empty list

Write Operation:
  Save to Hive  (always)
  Save to Firestore  (always)
```

4.3 Factory Pattern

Every model class provides a factory constructor fromMap() for deserializing data from both Hive and Firestore, and a copyWith() method for creating modified copies.

4.4 Stateful Widget State Management

No external state management library is used. Each screen manages its own local state via StatefulWidget — form controllers, loading flags, and list data.

5. Entry Point — main.dart

Initialization Flow

```
main() async
  ↓
WidgetsFlutterBinding.ensureInitialized()
  ↓
await Firebase.initializeApp()
  └─ Must run before any Firestore / Auth call
  ↓
await Hive.initFlutter()
  └─ Sets up local storage for offline fallback
  ↓
await DatabaseHelper.instance.initializeAdmin()
  └─ Seeds admin credentials in Hive + syncs to Firestore config/admin
  ↓
await DatabaseHelper.instance.initializeDefaultCategories()
  └─ Creates 5 default categories if none exist (Math, Science, ...)
```

```
  ↓
runApp(const QuizApp())
```

6. Database Layer — database_helper.dart

6.1 Dual-Storage Architecture

Hive Box	Firestore Collection	Purpose
questions	questions/	Quiz questions
categories	categories/	Quiz categories
students	students/	Student accounts
scores	scores/	Quiz results
admin	config/admin (doc)	Admin credentials

6.2 Admin Operations

initializeAdmin()

Called at startup. Checks Hive admin box — if empty, hashes 'admin123' and stores default credentials. Also syncs to Firestore config/admin using SetOptions(merge: true) so it won't overwrite a changed password.

verifyAdminCredentials(username, password)

Tries Firestore config/admin doc first. On failure falls back to Hive. Returns true only if both username and password hash match.

resetAdminPassword(newPassword)

Hashes the new password with SHA-256, then writes the new hash to both Hive and Firestore.

6.3 Question Operations

createQuestion(question)

Saves the question to both Hive 'questions' box and Firestore 'questions' collection using question.id as the key/doc-id.

getAllQuestions()

Tries Firestore first — fetches every document in 'questions' collection. Falls back to reading all values from Hive box.

getQuestionsByCategory(category)

Queries Firestore with a where('category', isEqualTo: category) filter. If Firestore is unavailable, calls getAllQuestions() and filters locally.

getQuestion(id)

Fetches a single specific question by its ID — first from Firestore, then Hive. Returns Question or null.

updateQuestion(question)

Delegates to createQuestion() — simply overwrites the existing document by the same ID in both stores.

deleteQuestion(id)

Deletes from Hive box by key and from Firestore collection by doc ID.

6.4 Category Operations

initializeDefaultCategories()

Calls getAllCategories() — if result is empty, creates 5 default categories (Math, Science, History, Geography, General Knowledge), saving each to both Firestore and Hive.

getAllCategories()

Firestore first from 'categories' collection, Hive fallback.

getAllCategoryNames()

Calls getAllCategories() then extracts and sorts the name strings. Used by dropdowns that only need names, not full objects.

getCategoryByName(name)

Calls getAllCategories() then searches case-insensitively. Returns the first match or null. Used for duplicate-name checking before create/update.

createCategory(category)

Checks for duplicate name via getCategoryByName(). If clear: saves to Hive and Firestore.

updateCategory(category, {String? oldName})

Updates the category in both Hive and Firestore. If oldName is provided and name has changed, calls updateQuestionsCategoryName() to cascade the rename to all linked questions.

updateQuestionsCategoryName(oldName, newName)

Fetches all questions for oldName, creates an updated copy of each with newName, and calls updateQuestion() for each — saving every change to both Firestore and Hive.

deleteCategory(id)

Deletes from Hive by key and Firestore by doc ID. CategoryManagementScreen guards against deletion when questions exist.

getQuestionCountByCategory(categoryName)

Calls getQuestionsByCategory() and returns the length. Used in CategoryManagementScreen to show question counts.

6.5 Student Operations

registerStudent(student, password)

1. Firebase Auth: createUserWithEmailAndPassword() — creates authenticated user. 2. Sends verification email. 3. Stores in Firestore with isActive: false and Firebase UID. 4. Stores in Hive with isActive: false. Account cannot be used until email is verified.

loginStudent(email, password)

1. Firebase Auth: signInWithEmailAndPassword(). 2. Reloads user token and checks emailVerified — throws ACCOUNT_NOT_ACTIVATED if false. 3. Updates Firestore student doc: isActive = true. 4. Returns Student from Firestore. Offline fallback: finds student by email in Hive, verifies password hash locally, checks isActive flag.

getStudentByEmail(email)

Hive-only lookup. Loops through local 'students' box comparing lowercase emails. Used in the offline login fallback path.

getStudentById(id)

Tries Firestore first. On failure (silent catch with empty catch block), reads from Hive by key. Returns Student or null.

6.6 Score Operations

saveScore(ScoreHistory)

Saves to Hive 'scores' box and Firestore 'scores' collection simultaneously.

getStudentScores(studentId)

Queries Firestore with where('studentId', isEqualTo: studentId), sorted by timestamp descending. Falls back to Hive — filters by studentId, sorts, returns.

7. Data Models

7.1 Student

Field	Type	Notes
id	String	Unique ID
name	String	Full name
email	String	Unique — used for Firebase Auth + login
address	String	Student address
phone	String	Phone number
passwordHash	String	SHA-256 hash — used for offline auth fallback
isActive	bool	true after email verification
createdAt	DateTime	Account creation timestamp

7.2 ScoreHistory

Field	Type	Notes
id	String	Unique ID
studentId	String	Foreign key linking to Student
category	String	Category name at time of quiz
score	int	Points scored
totalQuestions	int	Total questions in the quiz
timestamp	DateTime	When the quiz was taken

Computed property: percentage = (score / totalQuestions * 100).round()

7.3 Category

Field	Type	Notes
id	String	UUID (e.g. 'cat_1' for defaults, Uuid().v4() for new)
name	String	Category name — used as string reference in Questions

7.4 Question

Field	Type	Notes
id	String	Unique ID
questionText	String	The question displayed to the student
optionA / B / C / D	String	Four answer choices
correctAnswer	String	'A', 'B', 'C', or 'D'
category	String	Category name (string reference, not ID)

8. Utilities

8.1 PasswordUtils

Provides static SHA-256 hashing methods. Used for admin password storage and as the offline fallback verification method for student login.

hashPassword(password)

Converts plain text to UTF-8 bytes, applies SHA-256, returns hex string. One-way — cannot be reversed.

verifyPassword(password, hashedPassword)

Hashes the plain-text input and compares to stored hash. Returns true only if equal.

8.2 SessionManager

Singleton that persists login state in Hive's 'session' box. Both login screens call isAdminLoggedIn() / isStudentLoggedIn() in initState() for auto-login.

Method	Purpose
saveAdminSession()	Stores isAdminLoggedIn=true + login timestamp
isAdminLoggedIn()	Returns bool — used for admin auto-login check
logoutAdmin()	Clears admin session keys from Hive
saveStudentSession(id, name, email)	Stores all student session data
isStudentLoggedIn()	Returns bool — used for student auto-login check
getLoggedInStudentId()	Returns studentId from session or null
logoutStudent()	Clears student session keys from Hive
clearAllSessions()	Clears both admin and student sessions

9. Screens — Admin Section

HomeScreen

StatelessWidget landing page. Two buttons: 'Administrator' → AdminLoginScreen and 'Start Quiz' → StudentLoginScreen.

AdminLoginScreen

_checkExistingSession()

Called in initState(). Checks SessionManager.isAdminLoggedIn() — if true, navigates directly to AdminDashboardScreen, skipping the login form.

_login()

Calls DatabaseHelper.verifyAdminCredentials() which checks Firestore config/admin first, then falls back to Hive. On success: saves session, navigates to AdminDashboardScreen.

_showForgotPasswordDialog()

Shows a dialog requiring master code 'ADMIN@RESET' plus new password confirmation. On success: calls resetAdminPassword() which updates both Hive and Firestore.

AdminDashboardScreen

StatelessWidget. Provides two navigation cards: 'Manage Categories' and 'Manage Questions'. Logout clears admin session and pops to HomeScreen.

CategoryManagementScreen

_loadCategories()

Calls getAllCategories() (Firestore → Hive). For each category, calls getQuestionCountByCategory() to populate the count map displayed under each item.

_deleteCategory(category)

Guards against deletion if question count > 0 (shows error, returns). Otherwise shows confirmation dialog, then calls deleteCategory(id) which removes from both Firestore and Hive.

AddCategoryScreen

_saveCategory()

Dual-mode form. In edit mode: skips duplicate check if name unchanged, then calls updateCategory(oldName: oldName) which triggers the cascade rename across all questions in both stores. In create mode: always checks for duplicates, generates UUID, calls createCategory() which saves to both Firestore and Hive. Pops with result=true to trigger list reload in parent.

QuestionManagementScreen

_loadQuestions()

Calls getAllQuestions() (Firestore → Hive). Groups questions into a Map<String, List<Question>> by category for display.

_deleteQuestion(id)

Calls deleteQuestion(id) which removes from both Firestore and Hive.

AddEditQuestionScreen

Dual-mode form. Loads category names from Firestore/Hive on init. Tapping an option circle marks it as the correct answer (highlighted green). On save: createQuestion() or updateQuestion() — both save to Firestore + Hive.

10. Screens — Student Section

StudentLoginScreen

_checkExistingSession()

If isStudentLoggedIn() is true, fetches student via getStudentById() (Firestore → Hive) and navigates to StudentDashboardScreen.

_login()

Calls loginStudent() which uses Firebase Auth. If ACCOUNT_NOT_ACTIVATED is thrown, shows verification prompt with resend option. On success: saves session and navigates to dashboard.

StudentRegistrationScreen

_register()

Calls registerStudent(student, password) which: (1) creates a Firebase Auth account, (2) sends a verification email, (3) stores the student in Firestore with isActive: false, (4) stores in Hive with isActive: false. Navigates to login screen — student must verify email before first login.

StudentDashboardScreen

Loaded with the Student object from login. Calls getStudentScores(student.id) on init — Firestore first, Hive fallback — displayed sorted newest first. Pass ≥50% shows green badge; fail <50% shows orange badge.

StudentCategorySelectionScreen

_startQuiz(category)

Calls getQuestionsByCategory(category) — Firestore first, Hive fallback. If no questions found, shows error toast. Otherwise navigates to StudentQuizScreen passing the questions list, student, and category name.

StudentQuizScreen

Manages currentQuestionIndex, score, selectedAnswer, and answered state. _answerQuestion() prevents re-answering and checks correctness. Correct answers show green; wrong answers show red with the correct answer highlighted. _nextQuestion() advances or navigates to StudentResultScreen on last question.

StudentResultScreen

_saveScore()

Called in initState() before build() renders. Creates a ScoreHistory object and calls saveScore() which writes to both Firestore and Hive simultaneously. 'Try Again' returns to category selection; 'Dashboard' returns to StudentDashboardScreen.

11. Database Schema

Firestore Document Structures

questions/{id}

```
{ id, questionText, optionA, optionB, optionC, optionD,
  correctAnswer, category }
```

students/{uid}

```
{ id, uid (Firebase Auth UID), name, email, address, phone,
  passwordHash, isActive, createdAt }
```

categories/{id}

```
{ id, name }
```

scores/{id}

```
{ id, studentId, category, score, totalQuestions, timestamp }
```

config/admin

```
{ username, passwordHash }
```

Data Relationships

```
Student (1) ──── (many) ScoreHistory
  student.id ──────────── scoreHistory.studentId

Category (1) ──── (many) Question
  category.name ─────────── question.category (string reference)

Note: Questions reference category by name string, not by ID.
This is why renaming a category must cascade to all questions.
```

12. Session Management

Sessions are stored in Hive's 'session' box, which survives app restarts. Both login screens check for an existing session in initState() and auto-navigate to the appropriate dashboard if found.

Session Key	Type	Purpose
isAdminLoggedIn	bool	Admin session flag
adminLoginTime	String (ISO8601)	Admin login timestamp
isStudentLoggedIn	bool	Student session flag

Session Key	Type	Purpose
studentId	String	Logged-in student ID
studentName	String	Display name (avoids re-fetching)
studentEmail	String	Display email (avoids re-fetching)
studentLoginTime	String (ISO8601)	Student login timestamp

13. User Workflows

Admin Workflow

```
HomeScreen → 'Admin' button
    ↓
AdminLoginScreen
    ├─ isAdminLoggedIn? YES → AdminDashboardScreen (auto-login)
    └─ NO → enter credentials
                ↓ verifyAdminCredentials() [Firestore → Hive fallback]
                ↓
AdminDashboardScreen
    ├─ Manage Categories
    │      ├─ Add → AddCategoryScreen → saveCategory() → Firestore + Hive
    │      ├─ Edit → updateCategory() → Firestore + Hive + cascade questions
    │      └─ Delete → only if 0 questions → Firestore + Hive
    └─ Manage Questions
           ├─ Add → createQuestion() → Firestore + Hive
           ├─ Edit → updateQuestion() → Firestore + Hive
           └─ Delete → deleteQuestion() → Firestore + Hive
```

Student Workflow

```
HomeScreen → 'Start Quiz' button
    ↓
StudentLoginScreen
    ├─ isStudentLoggedIn? YES → load from Firestore/Hive → Dashboard
    └─ NO:
         ├─ New student → StudentRegistrationScreen
         │    → Firebase Auth account + verification email
         │    → Firestore (isActive: false) + Hive
         │    → must verify email before login
         └─ Existing → Firebase Auth login
              → check emailVerified
              → update isActive: true in Firestore
              ↓
        StudentDashboardScreen
              → score history from Firestore/Hive
              → 'Start Quiz' → category selection
                    → questions from Firestore/Hive
                    → quiz → answer all questions
                    → StudentResultScreen
                         → score saved to Firestore + Hive
                         → 'Try Again' | 'Dashboard'
```

14. Data Flow Diagrams

Category Rename with Cascade

```
Admin renames 'Maths' → 'Mathematics'
    ↓
DatabaseHelper.updateCategory(newCategory, oldName: 'Maths')
    ├─ Update Hive 'categories' box
    ├─ Update Firestore 'categories' doc
    └─ oldName provided → updateQuestionsCategoryName()
            ↓
        getQuestionsByCategory('Maths') [Firestore → Hive]
            ↓
        For each question:
          create copy with category = 'Mathematics'
          updateQuestion() → Hive + Firestore
    ↓
All questions now reference 'Mathematics' in both stores
```

Student Registration + Email Verification

```
StudentRegistrationScreen._register()
    ↓
Firebase Auth: createUserWithEmailAndPassword(email, password)
    ↓
sendEmailVerification()
    ↓
Firestore 'students' doc: { ...fields, isActive: false, uid }
Hive 'students' box:      { ...fields, isActive: false }
    ↓
Navigate to login → 'Check your email' message
    ↓
Student clicks link in inbox
    ↓
Student logs in → Firebase Auth checks emailVerified == true
    ↓
Firestore student doc updated: isActive = true
    ↓
Student accesses dashboard
```

Score Save Flow

```
StudentQuizScreen: last question answered
    ↓
Navigate to StudentResultScreen(score, total, student, category)
    ↓
initState() → _saveScore()
    ↓
Create ScoreHistory { studentId, category, score, total, timestamp=now }
    ↓
DatabaseHelper.saveScore()
    ├─ Hive 'scores' box
    └─ Firestore 'scores' collection
    ↓
UI renders result
    ↓
Score appears in dashboard history on next load
```

15. Navigation Flow

```
HomeScreen
├── AdminLoginScreen (session check → auto-login or form)
│       └── AdminDashboardScreen
│               ├── CategoryManagementScreen
│               │       ├── AddCategoryScreen (add)
│               │       └── AddCategoryScreen (edit, category param)
│               └── QuestionManagementScreen
│                       └── AddEditQuestionScreen (add / edit)
│
└── StudentLoginScreen (session check → auto-login or form)
        ├── StudentRegistrationScreen (new account)
        └── StudentDashboardScreen
                └── StudentCategorySelectionScreen
                        └── StudentQuizScreen
                                └── StudentResultScreen
                                        ├── Try Again → categories
                                        └── Dashboard → student dashboard

All screens: Home icon → HomeScreen (removeUntil clears entire stack)
```

Key Navigation Patterns

Data return (list refresh trigger)

```
final result = await Navigator.push(context, ...);
if (result == true) _loadCategories(); // reload after add/edit
```

Clear stack (login / logout)

```
Navigator.pushAndRemoveUntil(context,
  MaterialPageRoute(builder: (_) => HomeScreen()),
  (route) => false,
);
```

16. Summary

Layer	Files	Purpose
Entry Point	main.dart	Firebase + Hive init, default data seeding
Database	database_helper.dart	All CRUD — Firestore primary, Hive fallback
Models	student.dart, category.dart, question.dart	Data structures with toMap / fromMap
Utilities	session_manager.dart, password_utils.dart	Hive sessions + SHA-256 hashing

Layer	Files	Purpose
Admin Screens	home, admin_login, admin_dashboard, category_management, add_category, question_management, add_edit_question	Full admin panel
Student Screens	student_login, student_registration, student_dashboard, student_category_selection, student_quiz, student_result	Full student experience

Core Architecture Principles

- Dual Storage: every write hits both Firestore (cloud) and Hive (local)
- Online-First Reads: Firestore first, silent fallback to Hive on any error
- Firebase Auth: student accounts and email verification handled by Firebase
- Singleton Access: DatabaseHelper.instance and SessionManager.instance throughout
- Session Persistence: Hive 'session' box for auto-login across app restarts
- SHA-256 Hashing: admin passwords and offline student password verification
- Cascade Consistency: renaming a category automatically updates all linked questions in both stores